

1. Introduction to AI

Overview of Artificial Intelligence, Machine Learning, and Deep Learning

- **Artificial Intelligence (AI):**
The science of building systems that can perform tasks requiring human intelligence (e.g., reasoning, learning, decision-making).
 - Example: Google Maps suggesting the fastest route.
- **Machine Learning (ML):**
A subset of AI where systems learn patterns from data instead of being explicitly programmed.
 - Example: Netflix recommending movies based on your viewing history.
- **Deep Learning (DL):**
A subset of ML using multi-layered neural networks for complex tasks like image recognition, speech, or natural language.
 - Example: Face unlock on smartphones.

2. Introduction to Generative AI and its Significance

- **Generative AI (GenAI):** AI models that **create new content** (text, images, music, code) rather than just analyzing existing data.
 - Example: ChatGPT writing an essay, DALL·E creating an artwork.

Significance:

- Creativity support (design, music, story-writing).
 - Productivity boost (drafting emails, code generation).
 - Scientific research (drug discovery by generating new molecular structures).
-

3. Gen AI Landscape: Discriminative vs. Generative Modeling

- **Discriminative Models:** Learn boundaries between classes.
 - Example: A spam filter (spam vs. not spam).
- **Generative Models:** Learn the distribution of data and can **generate new samples**.
 - Example: A model generating realistic fake email texts.

Analogy:

- Discriminative = Judge deciding if food is Indian or Italian.
 - Generative = Chef who can cook Indian or Italian dishes from scratch.
 -
-

4. Information Theory Essentials

- **Entropy:** A measure of uncertainty in data.
 - Example: Tossing a fair coin (50-50) → high uncertainty. Tossing a two-headed coin → zero uncertainty.
 - **Mutual Information:** Measures how much knowing one variable reduces uncertainty about another.
 - Example: Weather forecast and umbrella sales – strong mutual information.
-

5. Variational Inference (High-Level)

- A method to approximate complex probability distributions using simpler ones.
 - Example: Instead of mapping all possible ways a student might fail/succeed in an exam (too complex), approximate it with a bell curve for simplicity.
-

6. Amortized Inference

- Re-using a trained model to make inference faster instead of solving optimization every time.
 - Example: Instead of solving a math problem from scratch each time, you memorize a formula (trained neural network) and apply it quickly.
-

7. Evaluation of Generative Models

- **Log-likelihood:** Probability the model assigns to real data.
 - Example: Language model predicting “I love AI” vs. “AI love I.”
 - **FID (Fréchet Inception Distance):** Measures how close generated images are to real ones.
 - Example: Checking how realistic AI-generated cats look compared to real cats.
 - **IS (Inception Score):** Measures diversity + quality of generated samples.
 - Example: Model generates dogs of different breeds, not just one type.
 - **PRD (Precision-Recall for Distributions):** Balances quality and variety.
 - **Human Evaluation:** Asking people to judge if outputs are realistic/useful.
 - Example: Turing test for text generation.
-

8. Core Components: Neural Networks, Embeddings, and Representation Learning

- **Neural Networks:** Modelled after the human brain, used for pattern recognition.
 - Example: Instagram automatically tagging friends in photos.
 - **Embeddings:** Converting data (words, images) into numerical vectors.
 - Example: “King – Man + Woman $\approx$ Queen” in word embeddings.
 - **Representation Learning:** Learning useful features automatically.
 - Example: Instead of manually coding "two eyes, a nose, mouth" to detect faces, the network learns these patterns itself.
-

9. Data, Tokenization, Embeddings, Conditioning Signals

- **Data:** The fuel for GenAI (text, images, audio).
- **Tokenization:** Splitting text into small pieces (words, subwords, characters).
 - Example: “ChatGPT is great” $\rightarrow$ [“Chat”, “G”, “PT”, “is”, “great”].
- **Embeddings:** Mapping tokens into dense vectors in high-dimensional space.
- **Conditioning Signals:** Extra info given to guide generation.
 - Example: Text-to-image model generating a "cat wearing sunglasses" – the prompt is the conditioning signal.

Real-life Example:

- When you ask Google Translate to translate “Hello” into French $\rightarrow$
 1. Tokenization: “Hello” $\rightarrow$ token.
 2. Embedding: Convert to vector.
 3. Conditioning: Target language = French.
 4. Generation: “Bonjour.”

1. Evaluation of Generative Models

When a generative AI creates something (text, image, music), **we need to check: Is it good or bad?**

That’s where evaluation metrics come in.

(a) Log-Likelihood

- **Idea:** How well does the model assign probability to real data?
- If the model gives **high probability** to correct sentences or images $\rightarrow$ it’s good.
- If it gives **low probability** $\rightarrow$ not good.

Example: Sentence: “I love ice cream.”

- Good model $\rightarrow$ thinks this is highly likely.
Sentence: “Ice cream love I.”
- Bad model $\rightarrow$ but if it thinks this is also likely, then it’s failing.

(b) FID (Fréchet Inception Distance)

- Used for **images**.
- Compares generated images with real ones.
- Lower FID = more realistic.

Example:

- You ask AI to generate dog images.
 - If FID is low → the dogs look natural (different breeds, sizes).
 - If FID is high → dogs look weird or fake (3 ears, no tail).
-

(c) IS (Inception Score)

- Measures **quality + diversity** of generated images.
- Good model = images are clear **and** varied.

Example:

- If AI makes 100 images of dogs:
 - All same-looking dogs → not diverse.
 - Dogs in different poses, colors, breeds → good diversity → higher IS.
-

(d) PRD (Precision-Recall for Distributions)

- Balances **precision** (realistic outputs) and **recall** (variety of outputs).

Example:

- Precision = making a few very realistic dogs.
 - Recall = making many dogs of all breeds, even if some look less real.
 - PRD ensures both are considered.
-

(e) Human Evaluation

- Sometimes metrics don't capture human judgment → we ask people!
- Humans check if text makes sense, or if an image looks natural.

Example:

- AI writes a story.
- Automatic metrics may say it's okay, but humans may find it boring → so human evaluation is very important.

2. Core Components of Generative AI

(a) Neural Networks

- A model inspired by the **human brain**.
- Made of “neurons” that process data and pass information.
- Can learn patterns automatically.

Example:

- Facebook detecting your friend’s face in a photo → neural networks learned patterns of eyes, nose, mouth.
-

(b) Embeddings

- AI cannot understand raw text/images → they are converted into **numbers (vectors)**.
- These number representations = **embeddings**.

Example (words):

- “King” and “Queen” → embeddings are close in meaning.
- “King – Man + Woman $\approx$ Queen.”
- AI uses embeddings to understand similarity.

Example (images):

- A cat image embedding is close to another cat image, but far from a car image.
-

(c) Representation Learning

- Instead of manually telling AI what features matter, AI **learns representations by itself**.
- Means AI discovers useful patterns automatically.

Example:

- Old method: To detect cats, humans coded “two ears, whiskers, tail.”
 - Representation learning: AI learns those features by looking at thousands of cat photos.
-

3. Data, Tokenization, Embeddings, Conditioning Signals

(a) Data

- The **fuel** of AI.
 - Can be text, images, audio, or video.
 - Quality of data = quality of AI.
 - **Example:**
If you train AI on medical books, it can help doctors. If trained on jokes, it won't know medicine!
-

(b) Tokenization

Breaking text into small pieces (tokens) that AI can process.

Example:

Sentence: "AI is amazing."

- Word tokenization → ["AI", "is", "amazing"]
- Subword tokenization → ["A", "I", "is", "amaz", "ing"]

Why? Because AI works better with numbers, not raw words.

(c) Embeddings (again, but in text)

- Tokens are turned into vectors (embeddings).
- This helps AI understand meaning.

Example:

"Happy" and "Joyful" will have embeddings close to each other.

(d) Conditioning Signals

- Extra information given to guide AI generation.

Types:

1. **Class labels** → "Generate a cat picture" (class = cat).
2. **Text prompts** → "Draw a cat wearing sunglasses."
3. **Masks** → Filling missing parts of an image or sentence.

Examples:

- Class: Generate digit “5” in handwritten style.
- Text: Generate a paragraph about cricket.
- Mask: Complete sentence → “I am ____ to school.” → AI predicts “going.”

1. What is Hugging Face?

- Hugging Face is a company + open-source community that provides **AI tools**.
- Its most famous library is **Transformers**, which gives access to thousands of **pretrained models** for tasks like:
 - Text generation (ChatGPT-like models)
 - Translation (English → French, etc.)
 - Sentiment analysis (positive/negative review)
 - Question answering (like asking a book questions)
 - Image & audio tasks (vision, speech recognition, etc.)

Real-life Example:

- Instead of training your own model from scratch (which takes months + huge GPUs), Hugging Face lets you **download a ready-made model in seconds** and use it right away.

2. What is the Transformers Library?

The **Transformers library** (by Hugging Face) is a collection of:

1. **Pretrained models** – GPT-2, BERT, T5, BLOOM, etc.
2. **Pipelines** – Easy functions to run tasks in one line of code.
3. **Tokenizers** – Break text into tokens (smaller units).
4. **Model Hub** – A website (<https://huggingface.co/models>) where thousands of models are shared.

What is Sentiment Analysis?

Sentiment Analysis is a technique in **Natural Language Processing (NLP)** that tries to understand the **feelings, emotions, or opinions** expressed in text.

In short, it answers: Is the text **positive**, **negative**, or **neutral**?

Module II: Core Generative Model Families

1. Variational Autoencoders (VAEs)

Idea: Teach a computer to **compress** data into a small “code” (latent space) and then **reconstruct** it back.

Components

- **Encoder** → Takes an input (image/text) and produces **mean (μ) and variance (σ^2)** of a probability distribution.
- **Latent space** → The compressed “idea” of the input.
- **Decoder** → Uses that latent code to reconstruct the original data.

Key Concepts

- **ELBO (Evidence Lower Bound):** The objective function VAEs maximize. Balances **reconstruction accuracy** and keeping latent codes close to a normal distribution.
- **β -VAE:** A modified version where a hyperparameter β controls the trade-off → higher β = more disentangled features (e.g., one latent dimension may represent “rotation,” another “thickness”).
- **Posterior collapse:** When the decoder ignores the latent code (happens if the decoder is too powerful).

Real-Life Example: Like summarizing a book into a few keywords (latent code). Then, from those keywords, reconstructing the whole story.

2. Generative Adversarial Networks (GANs)

Idea: A game between two neural networks:

- **Generator:** Creates fake data (e.g., fake images).
- **Discriminator:** Judges whether data is real or fake.

Key Concepts

- **Minimax game:** Generator tries to fool the discriminator; discriminator tries to catch the fakes.
- **Mode collapse:** Generator produces limited variety (e.g., only one type of face).
- **Wasserstein GANs (WGAN):** Use Wasserstein distance instead of cross-entropy → gives more stable training.
- **Conditional GANs (cGANs):** Generate data based on conditions (e.g., “generate a cat” vs “generate a dog”).
- **StyleGAN:** Advanced GAN for high-quality, controllable face generation.

Real-Life Example: Think of a **counterfeiter (generator)** printing fake currency and a **police officer (discriminator)** trying to detect fakes. Both get better over time.

3. Flow-Based Models

Idea: Learn an **invertible transformation** between simple distributions (like Gaussian noise) and complex data (like images).

Key Concepts

- **Normalizing Flows:** Step-by-step transformations that preserve probability.
- **Change of Variables Formula:** Used to compute exact likelihood.
- **Glow:** A popular flow-based model that can both generate and reconstruct images easily.

Real-Life Example: Like kneading dough (noise) into a complex bread shape (image), and being able to reverse it back into dough.

4. Energy-Based Models (EBMs)

Idea: Assign an “energy” (a scalar value) to each possible data configuration → lower energy = more likely.

- Models try to assign **low energy to real data** and **high energy to fake data**.
- Training involves **score matching** (computing gradients of log probability).
- Bridge to **diffusion models** (next-gen generative models like Stable Diffusion).

Real-Life Example: Imagine a landscape of hills and valleys. Real data lies in the **valleys (low energy)**, fake data in the **hills (high energy)**.

5. Trade-offs in Generative Models

Every family of models has **strengths & weaknesses**.

- **Likelihood vs. Sample Quality:**
 - VAEs → High likelihood, blurry samples.
 - GANs → Sharp samples, but low likelihood (hard to evaluate).
- **Stability:**
 - GANs are unstable to train (mode collapse, oscillation).
 - VAEs/flows more stable.
- **Compute:**
 - GANs/VAEs cheaper than diffusion models.
 - Flow models expensive (invertibility).

Real-Life Example: Like choosing between cameras:

- One gives **clear but few photos** (GAN),
- Another gives **lots of photos but slightly blurry** (VAE),
- Another can **recreate any scene exactly, but very slow** (Flow).

Model Type	Strengths	Weaknesses	Example Use Case
VAE	Stable, probabilistic, interpretable	Blurry images	Anomaly detection (medical images)
GAN	Sharp, realistic samples	Mode collapse, unstable	Deepfake videos, art generation
Flow-based	Exact likelihood, invertible	Expensive	Lossless image compression
Energy-based	General framework	Hard to train	Bridge to diffusion models

1. The Problem

In a VAE, we want to compute the probability of data (like images of digits):

$$p(x) = \int p(x|z)p(z)dz$$

But this integral is **too hard** to compute because it requires summing over all possible latent variables z .

Example: If I say “what’s the chance this photo is a cat?”, you’d have to imagine **all possible shapes, sizes, and colors of cats** → impossible to calculate directly.

2. The Trick: Approximation

Instead of computing the real (hard) posterior $p(z|x)$, we approximate it with a simpler distribution $q(z|x)$.

- True distribution: too complex.
- Approximation: easier to work with.

Example: Instead of considering all cat variations, you approximate with a simpler rule like “most cats are small, furry, and have pointy ears.”

3. The ELBO Formula

We maximize **ELBO**:

$$\text{ELBO} = \mathbb{E}_{q(z|x)} [\log p(x|z)] - KL(q(z|x) || p(z))$$

This has **two parts**:

1. **Reconstruction term** → make the output look like the input.
 - (How well the decoder rebuilds the original image/text).
2. **Regularization term (KL Divergence)** → keep the latent codes simple, like a normal distribution.
 - (Prevents overfitting and keeps the latent space smooth).

3. • If we only cared about reconstruction → we'd get overfitting (model memorizes training data).
4. • If we only cared about KL divergence → reconstructions would be blurry.
5. • **ELBO balances:** good reconstructions + nice, general latent space.

1. GANs (Generative Adversarial Networks)

Idea: A *Generator* (G) tries to create fake data, while a *Discriminator* (D) tries to tell fake from real.

It's like a **counterfeiter vs. police** game.

- **Minimax game:**
 - Generator improves to fool the discriminator.
 - Discriminator improves to catch fakes.
 - Training = competition until Generator becomes good at making realistic samples.
 - **Real-life example:** A student (generator) writes a fake essay, teacher (discriminator) checks if it's original. Over time, both improve.

GANs (Generative Adversarial Networks)

What it is:

- A **Generator (G)** makes fake data (like fake images).
- A **Discriminator (D)** tries to tell if data is *real or fake*.
- Training is like a **competition (minimax game)**:
 - G wants to fool D.
 - D wants to catch G.

Analogy:

- Think of a **counterfeiter (G)** making fake currency.
- The **police (D)** check the money.
- Over time:
 - Counterfeiter gets better at making realistic fakes.
 - Police get better at catching them.
- Eventually, fake money looks very real → this is what GAN achieves with images.

Why important?

- GANs can generate **photorealistic images**, art, deepfakes, and synthetic data for training AI.
-

2. StyleGAN (Special GAN)

What it adds to GANs:

- Introduces **style control** at different levels.
 - Coarse features → face shape, pose.
 - Medium features → hair style, eyes.
 - Fine features → freckles, wrinkles.

Analogy:

- Like a **photo editor** where you can adjust sliders:
 - Slider 1: Face shape.
 - Slider 2: Smile.
 - Slider 3: Hair.

Famous use:

- Generated hyper-realistic faces in "This Person Does Not Exist" website.

Student Note: StyleGAN = GAN + style control → gives you editing powers for generated images.

3. Flow-based Models

Core idea:

- Start with something simple (like random noise from a Gaussian distribution).
- Apply a series of **invertible transformations** to turn it into complex data (like a realistic image).
- Because transformations are invertible, we can go both ways:
 - Noise → Image (generation).
 - Image → Noise (analysis).

Analogy:

- Think of **dough shaping**: start with a round ball of dough (simple), stretch, fold, and twist → croissant (complex).
 - You can also “unfold” back to the dough.
-

Normalizing Flows

- "Normalizing" = start with a normal (Gaussian) distribution.
- "Flows" = apply flows (transformations) step by step.
- Allows **exact probability calculation** of data → something GANs cannot do.

Glow (Example of Flow-based Model)

- Glow is a famous flow-based model.
- It produces **high-quality images** and allows easy **latent space interpolation**.

Analogy:

- Imagine taking a selfie and smoothly morphing it into another face → Glow makes this possible.

4. Energy-Based Models (EBMs)

Concept:

- Each data point (image, digit, etc.) has an **energy score**.
- **Realistic samples = low energy**.
- **Fake samples = high energy**.
- Model learns to assign lower energy to good data.

Analogy:

- A ball rolling on a landscape:
 - Deep valleys = realistic images.
 - Hills = fake data.
 - Ball (training process) rolls towards valleys (better samples).

Score Matching (Bridge to Diffusion Models)

- Score = gradient of probability distribution (tells us the direction towards “more real” data).
- Diffusion models (like Stable Diffusion, DALL·E 2) use this idea:
 - Start from pure noise.
 - Iteratively denoise it using learned scores.

Analogy:

- Imagine restoring an old blurry photo step by step → each step guided by “what a clear photo should look like.”

Trade-offs in Generative Models

No model is perfect. Each has pros and cons:

Model	Strengths	Weaknesses
GANs	Very realistic images, fast	Hard to train, mode collapse
StyleGAN	Control over style, high quality	Complex, high compute
Flow Models	Exact probabilities, invertible	Slower, quality not as high as GANs
EBMs	Theoretical foundation, bridge to diffusion	Hard optimization
Diffusion Models	Best quality, stable	Very slow to generate

Key Trade-offs:

- 1. Likelihood vs. Sample Quality**
 - Flow models → exact likelihood, but medium-quality samples.
 - GANs → amazing quality, but no likelihood estimate.
- 2. Stability vs. Complexity**
 - GANs → unstable training.
 - Diffusion → stable but heavy compute.
- 3. Compute vs. Speed**
 - GANs → fast generation.
 - Diffusion → slow but high quality.

Key Trade-offs:

- 1. Likelihood vs. Sample Quality**
 - Flow models → exact likelihood, but medium-quality samples.
 - GANs → amazing quality, but no likelihood estimate.
- 2. Stability vs. Complexity**
 - GANs → unstable training.
 - Diffusion → stable but heavy compute.
- 3. Compute vs. Speed**
 - GANs → fast generation.
 - Diffusion → slow but high quality.